UNIVERSITY OF MINNESOTA · SUPPLEMENTARY MODULE

Sharding Across Azure Storage Accounts

When to shard, how to know, how it works, and what it does and does not buy you.

Azure Storage Architecture for Multi-Petabyte Portfolios



The account is the unit of quota

Capacity, request rate, ingress, and egress targets are enforced per account.

Sharding splits a portfolio across accounts so no single account's quota, policy set, or blast radius becomes the constraint.

On a frame you added shelves and controllers; in Azure you add accounts.

Everything else is deciding where the cut lines go.



Per-account targets (standard accounts, defaults)

5 PiB

Capacity

Higher by request.

40,000/s

Request rate

Major regions; 20,000 elsewhere.

250

Accounts per region per subscription

By default. [confirm current increase limit on Microsoft Learn]

60 Gbps

Ingress

Major regions; 25 Gbps elsewhere.

200 Gbps

Egress

Major regions; 50 Gbps elsewhere.

High-end targets, not guarantees; achieved rates depend on object size and access pattern. Source: Microsoft Learn.

What an account decides once, for everything inside it



Redundancy

LRS, ZRS, GRS, GZRS, and read-access variants.



Hierarchical namespace

On or off.



Performance tier

Standard or premium.



Access and network

Shared key setting and network posture (firewall, private endpoints).



NFS 3.0 or SFTP

Enabled rules out immutability policies.



Scope

Keys, account-level RBAC, lifecycle policy, cost records, deletion scope.

Three triggers, in priority order

1 Policy conflict

FORCED

2 Blast radius and administration

FORCED

3 Scale

A JUDGMENT CALL

Students expect sharding to be about size; most real splits come from the first two.

TRIGGER 1

Policy conflict

Datasets needing different account-level answers cannot share an account.

Archive tier

Requires LRS, GRS, or RA-GRS; an active corpus that wants ZRS goes elsewhere.

Hierarchical namespace

Gives up versioning, change feed, object replication, point-in-time restore, and version-level WORM.

NFS 3.0 or SFTP

Rules out immutability on that account.

The most common reason to shard, and the one most often missed.



TRIGGER 2

Blast radius and administration

- An account scopes its keys, RBAC, private endpoints, lifecycle policy, cost records, and deletion
- A compromised key or mistaken delete reaches everything in one account and nothing outside it
- An immutable backup target never shares an account with the data it protects
- A research unit that bills separately gets cleaner showback from its own accounts
- Separation of duties maps to separate accounts, often separate subscriptions

TRIGGER 3

Scale

Measured or projected demand approaches an account target. Capacity is easy to see coming from inventory growth rate; request rate and bandwidth depend on access pattern, not size.

50 TB

Millions of small objects

Can exhaust a request-rate target.

2 PB

Large files

Never touches it.

How you know: four signals in Azure Monitor



Used capacity

Against 5 PiB, with growth rate from the inventory.



Transactions, Ingress, Egress

Metrics against per-region targets.



Throttling

ResponseType on Transactions showing 503 Server Busy or 500 Operation Timeout.



Policy and ownership

Questions that have nothing to do with metrics.

Practitioner rule (not a Microsoft figure): plan the split when a projection crosses **60 to 70 percent** of a target.

Reading the signals

Signal	Response
Any policy question answered yes	**Shard**
Throttling appears	Fix naming first, then isolate the noisy workload
Capacity or bandwidth trends toward a target	Shard by key range before you reach it

Throttling fires at the partition level, so an account well under its totals can still throttle on one hot name range.

A sharding scheme is three things

1	2	3
A key	**A map**	**Consequences**
The attribute that decides which account a dataset lands in: data domain, protection class, request profile.	What answers "which account holds X": a deterministic naming convention, a catalog, or both.	Nothing is atomic across accounts.

Worked example: the inventory workbook's sharded account name column, where account is a deterministic function of research unit and data class.

The consequences of cutting

What you lose

No cross-account listing, lifecycle rule, or immutability policy.

Each account has its own keys, RBAC, private endpoints, lifecycle policy, and cost records.

What it costs

Moving data between accounts is a copy (AzCopy, object replication, or Data Factory), billed as transactions and, across regions, as egress.

Automation makes the tenth account as cheap to govern as the first: Bicep per data class, tags for showback.

How the virtual maps to the physical

- The physical unit is the storage stamp: racks of storage nodes in one datacenter, hosting many accounts from many customers
- Your account is assigned to a stamp in your region (three zone stamps for ZRS); its name becomes a DNS entry routing to that stamp
- From then on the account is a logical boundary only
- Three layers between a request and a disk: front end, partition, stream



Inside the stamp

1. **Front end**

   Authenticates and routes requests.

2. **Partition layer**

   Partition key is account plus container plus blob name; keys group into range partitions served by one partition server at a time; ranges split, merge, and move with load.

3. **Stream layer**

   Bytes stored as extents, each replicated three times across separate fault and upgrade domains.

A large blob is many extents on many nodes; moving a range partition moves no data.

What this means for sharding

ANALOGY

Stamp

The frame

Account

A volume with a QoS policy and its own keys

Partition server

The controller that owns the range right now

Quotas, not hardware

Two accounts in one region may share a stamp: sharding buys independent quotas and blast radius, not hardware isolation.

Only region, zone redundancy, and performance tier influence physical placement.

Hot spots stay inside

The hot-partition problem lives inside a shard: adjacent names land on one partition server until the platform rebalances.

A single blob has its own targets; sharding never helps a workload that hammers one object.



Naming is sharding one level down

- Partitioning is range based on the full blob name
- Timestamps, sequential IDs, and date-first prefixes cluster writes on one partition server
- A hash prefix spreads a high-rate write pattern across range partitions
- Sequential names are fine for archival data written once and listed by range
- **Choose the convention before the first petabyte lands; renaming is a copy**

The techniques, as a menu

Cut	How	Note		
By data domain			One account or set per research unit or project	The default cut, cleanest ownership and showback
By protection class	Separate accounts per redundancy, namespace, and immutability combination	Forced by Trigger 1		
By request profile	Isolate high-transaction workloads from bulk	Premium block blob when latency is the requirement		
By key range or hash	Year, instrument, or dataset ID modulo N	The only technique really about size, and the last one you reach for		
Within an account	By naming	The same concept applied to blob names		

Two clarifications and a ten-second rule

Clarifications

- Containers are not shards: namespace, RBAC scope, and container-level immutability, but no quota
- Geo-redundancy is not sharding: GRS copies the whole account to a fixed paired region; sharding is you choosing the cut lines

The ten-second rule

- Policy conflict, different owner, or different protection need: separate accounts
- Throttling: fix naming, then isolate the noisy workload
- Capacity or bandwidth trending toward a target: split the domain by key range first
- Everything else stays together; fewer accounts are easier to govern, and Bicep makes the ones you need cheap

